# Istio Service Mesh Workshop

2026-04-21

# Introduction

Welcome to the Istio Service Mesh Workshop. This workshop is designed to help you understand, deploy, and manage Istio service mesh in your Kubernetes environment. Through hands-on exercises and detailed explanations, you will learn how to leverage Istio's powerful features to enhance the security, observability, and reliability of your microservices architecture.

## Prerequisites

This workshop assumes the following:

- Basic understanding of Kubernetes concepts (pods, services, deployments)
- Familiarity with containerization and microservices architecture
- Access to a Kubernetes cluster
- Command-line experience with `oc` (OpenShift CLI)

If you are new to service meshes or Istio, we recommend reviewing the official Istio documentation before proceeding.

## Workshop Structure

This workshop is divided into four modules:

- **Module 1: Overview** - Introduction to Istio, Envoy, and sidecar architecture. Learn about the benefits of Istio, the strengths of Envoy proxy, and the basics of sidecar architecture. Hands-on exercises cover enabling sidecar injection and configuring mutual TLS (mTLS).
- **Module 2: Traffic Management** - Learn how traffic enters the mesh through Gateways and how VirtualServices route traffic within the mesh. Exercises include creating Gateways, implementing path-based routing, weighted traffic splitting for canary deployments, and combining Gateways with VirtualServices.
- **Module 3: Advanced Traffic Management and Security** - Explore advanced DestinationRule features including load balancing, failover, and circuit breaking. Learn how to implement request authentication with JWT and create authorization policies for fine-grained access control.
- **Module 4: Observability** - Understand how Istio provides metrics, traces, and logs in standard formats. Learn to access Envoy metrics and logs directly from sidecar proxies, and explore observability data using built-in Istio tools without external dependencies.

## Learning Objectives

After completing this workshop, you will be able to:

- Understand the core concepts and benefits of service mesh technology, including Istio and Envoy

- Enable sidecar injection and configure mutual TLS (mTLS) for secure service-to-service communication
- Create and configure Gateways to manage external traffic entry points
- Implement VirtualServices for sophisticated traffic routing, including path-based routing and weighted traffic splitting
- Configure DestinationRules for load balancing, failover, and circuit breaking
- Implement request authentication and authorization policies to secure your services
- Access and analyze metrics and logs directly from Envoy sidecar proxies
- Use Istio's built-in observability features to monitor and troubleshoot your service mesh
- Troubleshoot common issues and optimize Istio configurations

# Module 1: Overview

## Content

This module provides a comprehensive introduction to Istio service mesh, covering the fundamental concepts that form the foundation of service mesh technology. You will learn about the benefits of Istio, the strengths of Envoy proxy, and the basics of sidecar architecture.

### What is Istio?

Istio is an open-source service mesh platform that provides a uniform way to connect, secure, control, and observe microservices. It addresses the challenges of managing microservices by providing a dedicated infrastructure layer that handles service-to-service communication, security, and observability without requiring changes to application code.

### Benefits of Istio

Istio offers numerous advantages for organizations running microservices in Kubernetes environments:

#### Traffic Management

Istio provides sophisticated traffic management capabilities that enable you to:

- Control the flow of traffic and API calls between services
- Implement intelligent routing rules, including canary deployments and A/B testing
- Configure circuit breakers, timeouts, and retries to improve application resilience
- Perform traffic splitting and mirroring for gradual rollouts

#### Security

Security is a core strength of Istio:

- **Mutual TLS (mTLS)**: Automatic encryption of service-to-service communication
- **Authentication and Authorization**: Fine-grained access control policies
- **Certificate Management**: Automated certificate provisioning and rotation
- **Network Policies**: Enforcement of security policies at the network layer

#### Observability

Istio provides comprehensive observability without modifying application code:

- **Metrics**: Rich telemetry data for services, including request rates, latency, and error rates
- **Distributed Tracing**: End-to-end visibility into request flows across services
- **Access Logs**: Detailed logging of all service interactions

- **Service Topology**: Visual representation of service dependencies and communication patterns

#### Policy Enforcement

Istio enables you to enforce policies consistently across your microservices:

- Rate limiting and quota management
- Access control policies
- Network-level policies
- Compliance and governance enforcement

### Strengths of Envoy

Envoy is a high-performance, open-source edge and service proxy designed for cloud-native applications. Istio uses Envoy as its data plane proxy, and Envoy's architecture provides several key strengths:

#### Performance

- **High Throughput**: Envoy is written in C++ and optimized for performance, capable of handling millions of requests per second
- **Low Latency**: Minimal overhead added to service-to-service communication
- **Efficient Resource Usage**: Designed to be lightweight and resource-efficient

#### Advanced Features

Envoy provides a rich set of features that make it ideal for service mesh implementations:

- **Load Balancing**: Multiple algorithms including round-robin, least-request, ring hash, and more
- **Service Discovery**: Dynamic service discovery with support for multiple backends
- **Health Checking**: Active and passive health checking of upstream services
- **Circuit Breaking**: Automatic failure detection and circuit breaking
- **Retry Logic**: Configurable retry policies with exponential backoff

#### Observability

Envoy generates extensive telemetry data:

- **Access Logs**: Detailed HTTP/gRPC access logs
- **Metrics**: Prometheus-compatible metrics for monitoring
- **Distributed Tracing**: Built-in support for tracing with various backends

#### Extensibility

Envoy's architecture supports extensibility:

- **WebAssembly (WASM) Filters**: Custom logic can be added via WASM plugins
- **Lua Scripting**: Support for Lua-based custom filters
- **Plugin Architecture**: Modular design allows for custom extensions

## Sidecar Architecture Basics

The sidecar pattern is a fundamental architectural pattern used by Istio and other service mesh implementations. Understanding this pattern is crucial to understanding how service meshes work.

### What is a Sidecar?

A sidecar is a container that runs alongside your application container in the same pod. In Kubernetes, a pod can contain multiple containers, and the sidecar pattern leverages this capability to inject a proxy container (Envoy, in Istio's case) that handles all network communication for the application.

### How Sidecars Work

The sidecar architecture operates as follows:

1. **Application Container**: Your application runs in its primary container, making outbound requests and receiving inbound requests as it normally would.
2. **Sidecar Proxy**: The Envoy sidecar proxy intercepts all network traffic to and from the application container.
3. **Traffic Interception**: The sidecar uses iptables rules (or eBPF in newer implementations) to redirect traffic through the proxy without the application being aware.
4. **Policy Enforcement**: The sidecar applies security policies, routing rules, and collects telemetry data based on configuration from the control plane.

### Benefits of the Sidecar Pattern

The sidecar architecture provides several advantages:

- **Transparency**: Applications don't need to be modified to benefit from service mesh features
- **Separation of Concerns**: Network and security logic is separated from business logic
- **Consistency**: All services use the same proxy, ensuring consistent behavior
- **Independent Updates**: The sidecar can be updated independently of the application

### Sidecar Injection

Istio uses automatic sidecar injection to add the Envoy proxy to pods:

- **Mutating Webhook**: Kubernetes admission controller automatically injects the sidecar when pods are created
- **Namespace Labeling**: Pods in namespaces labeled with `istio-injection=enabled` automatically receive sidecars

- **Manual Injection**: Sidecars can also be injected manually using the `istioctl` command

### Summary

In this module, we quickly reviewed the following concepts:

- Istio is a powerful service mesh platform that provides traffic management, security, and observability for microservices
- Envoy proxy offers high performance, advanced features, and extensive observability capabilities
- The sidecar architecture pattern enables transparent network management without modifying applications
- Istio's sidecar injection mechanism automatically adds Envoy proxies to your pods

In the next module, you will learn how to install and configure Istio in your Kubernetes cluster.

## Exercises

This exercise section provides hands-on practice with OpenShift Service Mesh 3. You will learn how to enable sidecar injection and configure mutual TLS (mTLS) for your applications.

### Prerequisites

Before starting these exercises, ensure you have:

- Access to an OpenShift cluster with Service Mesh 3 installed
- Cluster administrator or project administrator permissions
- The `oc` CLI tool installed and configured
- A project namespace where you can deploy applications

### Exercise 1: Enable Sidecar Injection

In this exercise, you will enable automatic sidecar injection for a namespace in OpenShift Service Mesh 3.

#### Understanding Istio CNI

Before verifying the installation, it's important to understand the role of Istio CNI (Container Network Interface) in the service mesh architecture.

Istio CNI is a critical component that handles network configuration for pods in the service mesh. Its primary purpose is to:

- **Traffic Redirection**: Sets up rules to route pod traffic through the Envoy sidecar without needing extra privileges.
- **Security**: Removes the need for `NET_ADMIN` and `NET_RAW`, making pods safer.

- **Transparency**: Automatically intercepts all pod traffic.
- **Initialization**: Prepares networking before the app starts.

Without Istio CNI, sidecar injection would require pods to run with elevated privileges. The CNI plugin configures networking securely at pod startup.

#### Step 1: Verify Service Mesh Installation

First, verify that OpenShift Service Mesh 3 is installed and all critical components are running. Check the Istio control plane pods:

```
oc get pods -n istio-system
```

You should see the Istio control plane components running, including `istiod` pods. Verify they are in `Running` state:

```
oc get pods -n istio-system -l app=istiod
```

Next, verify that Istio CNI is installed and running. The CNI components are deployed in the `istio-cni` namespace as DaemonSets to run on each node:

```
oc get pods -n istio-cni
```

You should see Istio CNI pods running on each node in your cluster. Verify they are in `Running` state:

```
oc get daemonset -n istio-cni
```

#### Step 2: Set Your Namespace

Set the `NAMESPACE` environment variable to your existing sandbox namespace. Replace `{USER}` with your actual username:

```
export NAMESPACE={USER}-sandbox
```

For example, if your username is `jdoe`, your namespace would be `jdoe-sandbox`:

```
export NAMESPACE=jdoe-sandbox
```

Verify that the namespace exists:

```
oc get namespace $NAMESPACE
```

#### Step 3: Enable Sidecar Injection

Enable automatic sidecar injection by adding the revision label to your namespace. The revision label specifies which Istio control plane revision to use for sidecar injection. For Istio version 1.23, use:

```
oc label namespace $NAMESPACE istio.io/rev=default-1.23.0
```

> The revision value should match your Istio installation revision. This example uses `default-1.23.0` for Istio version 1.23.

#### Step 4: Verify Sidecar Injection Label

Verify that the revision label has been applied:

```
oc get namespace $NAMESPACE --show-labels
```

You should see `istio.io/rev=default-1.23.0` in the labels list.

#### Step 5: Deploy a Test Application

Deploy a simple application to verify that sidecar injection is working:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-app
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: test-app
  template:
    metadata:
      labels:
        app: test-app
    spec:
      containers:
      - name: test-container
        image: nginx:latest
        ports:
        - containerPort: 80
EOF
```

#### Step 6: Verify Sidecar Injection

Check that the Envoy sidecar has been injected into your pod:

```
oc get pods -n $NAMESPACE -l app=test-app
```

Describe the pod to see all containers:

```
oc describe pod -n $NAMESPACE -l app=test-app | grep -A 10 "Containers:"
```

You should see two containers: your application container and the `istio-proxy` sidecar container.

### Exercise 2: Enable Mutual TLS (mTLS)

In this exercise, you will configure mutual TLS for your namespace using OpenShift Service Mesh 3.

#### Step 1: Create a PeerAuthentication Policy

Create a PeerAuthentication resource to enable strict mTLS for your namespace:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ${NAMESPACE}
spec:
  mtls:
    mode: STRICT
EOF
```

This policy enforces mTLS for all traffic within your namespace.

#### Step 2: Verify PeerAuthentication Policy

Verify that the PeerAuthentication policy has been created:

```
oc get peerauthentication -n $NAMESPACE
```

#### Step 3: Create a DestinationRule (Optional but Recommended)

While PeerAuthentication enables mTLS, it's a best practice to also create a DestinationRule to explicitly specify the TLS mode:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
```

```
kind: DestinationRule
metadata:
  name: default
  namespace: ${NAMESPACE}
spec:
  host: "*.${NAMESPACE}.svc.cluster.local"
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
EOF
```

This DestinationRule ensures that all services in the namespace use Istio-managed certificates for mTLS.

**Step 4: Verify mTLS Configuration**

Check the Istio configuration to verify mTLS is enabled:

```
oc get peerauthentication -n $NAMESPACE -o yaml
oc get destinationrule -n $NAMESPACE -o yaml
```

**Step 5: Test mTLS Connection**

Deploy a second application to test mTLS communication:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-app-2
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: test-app-2
  template:
    metadata:
      labels:
        app: test-app-2
    spec:
      containers:
      - name: test-container
        image: curlimages/curl:latest
        command: ["sleep", "3600"]
EOF
```

Wait for the pod to be ready, then exec into it and test connectivity:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=test-app-2 -o
jsonpath='{.items[0].metadata.name}')
oc exec -n $NAMESPACE $POD_NAME -c test-container -- curl -v http://test-app:80
```

The connection should succeed, and you can verify in the Envoy access logs that mTLS is being used.

#### Step 6: View Envoy Access Logs

To verify mTLS is working, check the Envoy sidecar logs:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=test-app-2 -o
jsonpath='{.items[0].metadata.name}')
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy | tail -20
```

Look for log entries indicating TLS connections.

### Exercise 3: Verify mTLS Configuration

In this exercise, you will verify mTLS configuration using various methods.

#### Step 1: Check PeerAuthentication and DestinationRule Resources

Verify the mTLS policies are correctly applied by checking the resources:

```
oc get peerauthentication -n $NAMESPACE -o yaml
oc get destinationrule -n $NAMESPACE -o yaml
```

This shows the PeerAuthentication and DestinationRule configurations that enforce mTLS.

#### Step 2: Verify mTLS in Envoy Configuration

Check the Envoy sidecar configuration to verify mTLS is enabled. Get the name of a pod:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=test-app -o
jsonpath='{.items[0].metadata.name}')
```

If `istioctl` is available, you can check the proxy configuration:

```
istioctl proxy-config cluster $POD_NAME -n $NAMESPACE | grep -i tls
```

This shows the TLS settings for outbound clusters.

#### Step 3: Verify mTLS from Application Logs

Check the Envoy access logs to verify mTLS connections. The logs should show TLS protocol information:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=test-app-2 -o
jsonpath='{.items[0].metadata.name}')
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy | grep -i tls
```

You can also check for successful mTLS handshakes in the logs.

## Summary

In these exercises, you have:

- Enabled automatic sidecar injection for a namespace
- Verified that the Envoy sidecar is injected into application pods
- Configured mutual TLS (mTLS) using PeerAuthentication and DestinationRule resources
- Tested mTLS communication between services
- Verified mTLS configuration using `istioctl`

These foundational skills will be essential as you progress through the remaining modules of this workshop.

## Troubleshooting

If you encounter issues:

- **Sidecar not injecting**: Verify the namespace revision label is correct (`istio.io/rev=<revision>`) and matches your Istio installation revision. Ensure the Istio control plane is running.
- **mTLS not working**: Ensure both PeerAuthentication and DestinationRule are properly configured
- **Connection failures**: Check that both pods have sidecars injected and are in the same namespace (or that proper policies exist for cross-namespace communication)
- **Access denied**: Verify that the Istio control plane pods (`istiod`) are running and that certificates are being issued by checking the `istiod` logs

# Module 2: Traffic Management

## Content

This module explores how Istio manages traffic flow in your service mesh. You will learn how external traffic enters the mesh through Gateways, how internal traffic is routed using VirtualServices, and how these components work together to provide sophisticated traffic management capabilities.

### How Traffic Enters the Mesh: Gateways

A fundamental question when working with service meshes is: "How does traffic get onto the mesh?" In Istio, the answer is through **Gateways**.

#### What is a Gateway?

A Gateway is an Istio resource that configures a load balancer to handle incoming and outgoing HTTP/TCP traffic. Gateways operate at the edge of the service mesh, acting as the entry and exit points for traffic. They allow you to expose services to external clients and control how traffic enters and leaves your mesh.

Unlike traditional Kubernetes Ingress resources, Istio Gateways provide more sophisticated traffic management capabilities. They work in conjunction with VirtualServices to route traffic based on various criteria such as host headers, URI paths, and other HTTP attributes.

#### Gateway vs. OpenShift Routes

In OpenShift environments, you may be familiar with Routes, which provide external access to services. Istio Gateways serve a similar purpose but offer significantly more functionality:

- **Protocol Support**: Gateways support HTTP, HTTPS, gRPC, WebSocket, TCP, and TLS protocols, while Routes are primarily HTTP/HTTPS focused
- **Traffic Routing**: Gateways work with VirtualServices to provide advanced routing rules, including path-based routing, header-based routing, and weighted traffic splitting
- **Security Features**: Gateways integrate with Istio's security features, including mTLS termination, authentication, and authorization policies
- **Observability**: All traffic through Gateways is automatically instrumented with Istio's observability features
- **Multi-Protocol**: Gateways can handle multiple protocols on different ports simultaneously

While OpenShift Routes are simpler and sufficient for basic use cases, Gateways provide the foundation for advanced traffic management patterns that are essential in microservices architectures.

#### Gateway Configuration

A Gateway resource defines which ports to expose, the protocols to use, and which hosts are

allowed. Here's a basic example:

```
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
```

Key components of a Gateway:

- **Selector**: Specifies which Istio ingress gateway pods should handle this Gateway's traffic (typically `istio: ingressgateway`)
- **Servers**: Defines the ports, protocols, and hosts that the Gateway will accept
- **Hosts**: Specifies the destination hosts for incoming traffic (can use wildcards)

#### Gateway and Connectivity

Gateways are the primary mechanism for external connectivity in Istio. They enable:

- Ingress traffic from external clients to services within the mesh
- Egress traffic from services within the mesh to external services
- Multi-cluster connectivity (when configured with additional components)

For advanced connectivity scenarios, such as connecting multiple clusters or integrating with external service meshes, Istio provides additional connectivity features. These advanced connectivity topics, including multi-cluster setups and service mesh federation, are covered in specialized workshops and are beyond the scope of this module.

#### Gateway Best Practices

When configuring Gateways, consider the following best practices:

- Use specific host names rather than wildcards in production environments for better security
- Configure TLS termination at the Gateway for HTTPS traffic
- Use Gateway resources in combination with VirtualServices for complete traffic control
- Leverage Istio's security features to protect Gateway endpoints
- Monitor Gateway metrics and logs for traffic patterns and potential issues

## VirtualServices: Routing Traffic Within the Mesh

Once traffic enters the mesh through a Gateway (or originates from within the mesh), VirtualServices control how that traffic is routed to your services.

### What is a VirtualService?

A VirtualService is an Istio resource that defines a set of traffic routing rules to apply when a host is addressed. VirtualServices are a key component of Istio's traffic management, allowing you to configure how requests are routed to service versions.

Think of a VirtualService as a "virtual" representation of your service that defines the routing rules, while the actual Kubernetes Service represents the "real" service endpoints. This abstraction allows you to implement sophisticated routing patterns without modifying your application code.

### VirtualService Capabilities

VirtualServices provide powerful routing capabilities:

- **Path-based Routing**: Route traffic to different services based on the URI path
- **Header-based Routing**: Route traffic based on HTTP headers (useful for A/B testing, canary deployments)
- **Weighted Traffic Splitting**: Distribute traffic across multiple service versions with specified percentages
- **Fault Injection**: Introduce delays or errors for testing resilience
- **Request Matching**: Match requests based on various criteria (URI, headers, query parameters, etc.)
- **Traffic Mirroring**: Send a copy of traffic to another service without affecting the primary flow

### Basic VirtualService Example

Here's a simple VirtualService that routes all traffic to a service:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: my-service
spec:
  hosts:
  - my-service
  http:
  - route:
    - destination:
        host: my-service
        subset: v1
      weight: 100
```

This VirtualService routes all traffic for the `my-service` host to the `v1` subset of the service.

#### VirtualService with Path-based Routing

VirtualServices can route traffic to different services based on the request path:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: my-app
spec:
  hosts:
  - my-app.example.com
  http:
  - match:
    - uri:
        prefix: "/api/v1"
    route:
    - destination:
        host: api-v1-service
  - match:
    - uri:
        prefix: "/api/v2"
    route:
    - destination:
        host: api-v2-service
  - route:
    - destination:
        host: frontend-service
```

This example routes:
* Requests to `/api/v1/ to api-v1-service`
`* Requests to /api/v2/` to `api-v2-service`
* All other requests to `frontend-service`

#### VirtualService with Weighted Traffic Splitting

One of the most powerful features of VirtualServices is weighted traffic splitting, which is essential for canary deployments and gradual rollouts:

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: my-service
spec:
  hosts:
  - my-service
  http:
  - route:
```

```
    - destination:
        host: my-service
        subset: v1
      weight: 90
    - destination:
        host: my-service
        subset: v2
      weight: 10
```

This configuration routes 90% of traffic to version 1 and 10% to version 2, allowing you to gradually roll out a new version while monitoring its behavior.

#### VirtualService Matching Rules

VirtualServices support sophisticated matching rules to route traffic based on various request attributes:

- **URI Matching**: Match based on exact, prefix, or regex patterns
- **Header Matching**: Match based on HTTP headers (exact, prefix, regex, or presence)
- **Query Parameters**: Match based on query string parameters
- **Method Matching**: Match based on HTTP method (GET, POST, etc.)
- **Authority Matching**: Match based on the Host header
- **Port Matching**: Match based on the destination port

Multiple match conditions can be combined, and all must be satisfied for a match to occur.

#### VirtualService and DestinationRules

VirtualServices work closely with DestinationRules. While VirtualServices define **where** traffic should go, DestinationRules define **how** traffic should be handled when it reaches the destination:

- **Load Balancing**: Configure load balancing algorithms (round-robin, least-request, etc.)
- **Connection Pool Settings**: Configure connection pool sizes and timeouts
- **Circuit Breakers**: Configure circuit breaker settings for fault tolerance
- **Subsets**: Define service subsets (versions) for traffic routing
- **TLS Settings**: Configure TLS/mTLS settings for the destination

#### VirtualService Best Practices

When working with VirtualServices:

- Always define a default route (the last route without a match) to handle unmatched traffic
- Use meaningful names that reflect the service or application being routed
- Keep routing rules simple and maintainable
- Test routing rules in non-production environments first

- Use subsets defined in DestinationRules for version-based routing
- Monitor traffic patterns to validate routing behavior
- Document complex routing logic for team understanding

## Gateway and VirtualService Together

Gateways and VirtualServices work together to provide complete traffic management:

1. **Gateway**: Defines the entry point and which hosts/ports to accept
2. **VirtualService**: Defines the routing rules for traffic matching those hosts

When a request comes through a Gateway, the VirtualService matching the host header determines where the traffic is routed. This separation of concerns allows for flexible and powerful traffic management.

#### Example: Complete Ingress Configuration

Here's a complete example showing a Gateway and VirtualService working together:

```
# Gateway: Defines the entry point
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - myapp.example.com

---
# VirtualService: Defines routing rules
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: myapp
spec:
  hosts:
  - myapp.example.com
  gateways:
  - my-gateway
  http:
  - match:
    - uri:
```

```
        prefix: "/api"
    route:
    - destination:
        host: api-service
  - route:
    - destination:
        host: frontend-service
```

This configuration:
* Accepts traffic for `myapp.example.com` on port 80 through the Gateway
* Routes `/api/*` requests to `api-service`
* Routes all other requests to `frontend-service`

### Summary

In this module, you have learned:

- Gateways are the entry and exit points for traffic in the Istio service mesh
- Gateways provide more functionality than traditional Routes, including advanced routing, security, and observability
- VirtualServices define routing rules for traffic within the mesh
- VirtualServices support path-based routing, weighted traffic splitting, and sophisticated matching rules
- Gateways and VirtualServices work together to provide complete traffic management
- DestinationRules complement VirtualServices by defining how traffic is handled at the destination

In the next module, you will explore additional traffic management features and advanced routing patterns.

## Exercises

This exercise section provides hands-on practice with Istio traffic management. You will learn how to configure Gateways, create VirtualServices with various routing patterns, implement path-based routing, perform weighted traffic splitting, and combine these components for complete traffic control.

### Prerequisites

Before starting these exercises, ensure you have:

- Access to an OpenShift cluster with Service Mesh 3 (Istio >= 1.23) installed
- Your namespace (`$NAMESPACE`) configured with sidecar injection enabled (from Module 1)
- The `oc` CLI tool installed and configured
- Basic understanding of Kubernetes Services and Deployments

## Exercise 1: Create and Configure a Gateway

In this exercise, you will create a Gateway resource to allow external traffic into your service mesh.

### Step 1: Set Your Namespace

Ensure your namespace variable is set:

```
export NAMESPACE={USER}-sandbox
```

Verify the namespace exists and has sidecar injection enabled:

```
oc get namespace $NAMESPACE --show-labels | grep istio.io/rev
```

### Step 2: Verify Istio Ingress Gateway

Check that the Istio ingress gateway is running:

```
oc get pods -n istio-system -l app=istio-ingressgateway
```

Note the service name and external access method:

```
oc get svc -n istio-system istio-ingressgateway
```

### Step 3: Create a Basic Gateway

Create a Gateway that accepts HTTP traffic on port 80:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
  namespace: ${NAMESPACE}
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
EOF
```

#### Step 4: Verify Gateway Creation

Verify the Gateway was created successfully:

```
oc get gateway -n $NAMESPACE
```

View the Gateway configuration:

```
oc get gateway my-gateway -n $NAMESPACE -o yaml
```

#### Step 5: Create a Gateway with HTTPS

Create an additional Gateway that accepts HTTPS traffic on port 443:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-https-gateway
  namespace: ${NAMESPACE}
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 443
      name: https
      protocol: HTTPS
    tls:
      mode: SIMPLE
      credentialName: my-tls-secret
    hosts:
    - "myapp.example.com"
EOF
```

In a production environment, you would need to create the TLS secret referenced in the Gateway configuration.

## Exercise 2: Create VirtualServices with Basic Routing

In this exercise, you will create VirtualServices to route traffic to your services.

#### Step 1: Deploy Sample Applications

Deploy two sample applications to use for routing:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
      version: v1
  template:
    metadata:
      labels:
        app: frontend
        version: v1
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: ${NAMESPACE}
spec:
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 80
    name: http
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
      version: v1
  template:
    metadata:
```

```
      labels:
        app: backend
        version: v1
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: ${NAMESPACE}
spec:
  selector:
    app: backend
  ports:
  - port: 80
    targetPort: 80
    name: http
EOF
```

Wait for the pods to be ready:

```
oc get pods -n $NAMESPACE -l app=frontend
oc get pods -n $NAMESPACE -l app=backend
```

**Step 2: Create a Basic VirtualService**

Create a VirtualService that routes all traffic to the frontend service:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: frontend
  namespace: ${NAMESPACE}
spec:
  hosts:
  - frontend
  http:
  - route:
    - destination:
        host: frontend
      weight: 100
EOF
```

#### Step 3: Verify VirtualService

Check that the VirtualService was created:

```
oc get virtualservice -n $NAMESPACE
```

View the VirtualService configuration:

```
oc get virtualservice frontend -n $NAMESPACE -o yaml
```

#### Step 4: Test Internal Routing

Test that the VirtualService routes traffic correctly by sending a request from within the cluster:

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v http://frontend
```

### Exercise 3: Implement Path-Based Routing

In this exercise, you will create a VirtualService that routes traffic based on URI paths.

#### Step 1: Create a Path-Based VirtualService

Create a VirtualService that routes different paths to different services:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: myapp
  namespace: ${NAMESPACE}
spec:
  hosts:
  - myapp.example.com
  gateways:
  - my-gateway
  http:
  - match:
    - uri:
        prefix: "/api"
    route:
    - destination:
        host: backend
      weight: 100
  - match:
    - uri:
        prefix: "/"
```

```
    route:
    - destination:
        host: frontend
      weight: 100
EOF
```

This VirtualService:
* Routes requests to `/api/*` to the backend service
* Routes all other requests to the frontend service

**Step 2: Verify Path-Based Routing**

Check the VirtualService configuration:

```
oc get virtualservice myapp -n $NAMESPACE -o yaml
```

**Step 3: Test Path-Based Routing**

Get the ingress gateway external address:

```
INGRESS_HOST=$(oc get svc -n istio-system istio-ingressgateway -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
echo $INGRESS_HOST
```

If no external address is available, you can use port-forwarding:

```
oc port-forward -n istio-system svc/istio-ingressgateway 8080:80 &
```

Test the frontend route:

```
curl -H "Host: myapp.example.com" http://localhost:8080/
```

Test the backend API route:

```
curl -H "Host: myapp.example.com" http://localhost:8080/api
```

**Step 4: Create More Complex Path Matching**

Create a VirtualService with multiple path patterns:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
```

```
  name: complex-routing
  namespace: ${NAMESPACE}
spec:
  hosts:
  - complex.example.com
  gateways:
  - my-gateway
  http:
  - match:
    - uri:
        exact: "/api/v1/users"
    route:
    - destination:
        host: backend
      weight: 100
  - match:
    - uri:
        prefix: "/api/v2"
    route:
    - destination:
        host: backend
      weight: 100
  - match:
    - uri:
        regex: "^/static/.*\\.(jpg|png|gif)$"
    route:
    - destination:
        host: frontend
      weight: 100
  - route:
    - destination:
        host: frontend
      weight: 100
EOF
```

This example demonstrates:
* Exact path matching (`/api/v1/users`)
* Prefix matching (`/api/v2/*`)
* Regex matching (static image files)
* Default route (catch-all)

## Exercise 4: Implement Weighted Traffic Splitting

In this exercise, you will implement weighted traffic splitting for canary deployments.

### Step 1: Deploy Multiple Versions

Deploy a second version of the frontend application:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-v2
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
      version: v2
  template:
    metadata:
      labels:
        app: frontend
        version: v2
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
EOF
```

Wait for the new pods to be ready:

```
oc get pods -n $NAMESPACE -l app=frontend
```

**Step 2: Create DestinationRule with Subsets**

Create a DestinationRule to define service subsets (versions):

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: frontend
  namespace: ${NAMESPACE}
spec:
  host: frontend
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
```

```
EOF
```

Verify the DestinationRule:

```
oc get destinationrule -n $NAMESPACE
oc get destinationrule frontend -n $NAMESPACE -o yaml
```

**Step 3: Create VirtualService with Weighted Splitting**

Create a VirtualService that splits traffic 90% to v1 and 10% to v2:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: frontend-canary
  namespace: ${NAMESPACE}
spec:
  hosts:
  - frontend
  http:
  - route:
    - destination:
        host: frontend
        subset: v1
      weight: 90
    - destination:
        host: frontend
        subset: v2
      weight: 10
EOF
```

**Step 4: Test Weighted Traffic Splitting**

Send multiple requests and observe the traffic distribution. You can check the Envoy access logs to see which version receives each request:

```
for i in {1..20}; do
  oc run curl-test-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://frontend > /dev/null
done
```

Check the logs to see traffic distribution:

```
oc logs -n $NAMESPACE -l app=frontend,version=v1 -c istio-proxy --tail=10
```

```
oc logs -n $NAMESPACE -l app=frontend,version=v2 -c istio-proxy --tail=10
```

**Step 5: Adjust Traffic Weights**

Update the VirtualService to increase traffic to v2 (50/50 split):

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: frontend-canary
  namespace: ${NAMESPACE}
spec:
  hosts:
  - frontend
  http:
  - route:
    - destination:
        host: frontend
        subset: v1
      weight: 50
    - destination:
        host: frontend
        subset: v2
      weight: 50
EOF
```

**Step 6: Complete Canary Deployment**

Finally, route 100% of traffic to v2:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: frontend-canary
  namespace: ${NAMESPACE}
spec:
  hosts:
  - frontend
  http:
  - route:
    - destination:
        host: frontend
        subset: v2
      weight: 100
EOF
```

## Exercise 5: Combine Gateway and VirtualService

In this exercise, you will combine a Gateway and VirtualService to provide complete external access to your services.

### Step 1: Create a Complete Gateway Configuration

Ensure you have a Gateway that accepts traffic for your application:

```
oc get gateway my-gateway -n $NAMESPACE
```

If it doesn't exist, create it:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: my-gateway
  namespace: ${NAMESPACE}
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "myapp.example.com"
EOF
```

### Step 2: Create VirtualService for Gateway

Create a VirtualService that works with the Gateway to route external traffic:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: myapp-external
  namespace: ${NAMESPACE}
spec:
  hosts:
  - myapp.example.com
  gateways:
  - my-gateway
  http:
  - match:
```

```
    - uri:
        prefix: "/api"
    route:
    - destination:
        host: backend
        port:
          number: 80
      weight: 100
  - route:
    - destination:
        host: frontend
        port:
          number: 80
      weight: 100
EOF
```

**Step 3: Verify Gateway and VirtualService Integration**

Check both resources:

```
oc get gateway,virtualservice -n $NAMESPACE
```

View the complete configuration:

```
oc get gateway my-gateway -n $NAMESPACE -o yaml
oc get virtualservice myapp-external -n $NAMESPACE -o yaml
```

**Step 4: Test External Access**

Get the ingress gateway address:

```
INGRESS_HOST=$(oc get svc -n istio-system istio-ingressgateway -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
INGRESS_PORT=$(oc get svc -n istio-system istio-ingressgateway -o
jsonpath='{.spec.ports[?(@.name=="http2")].port}')
```

If using port-forwarding:

```
oc port-forward -n istio-system svc/istio-ingressgateway 8080:80 &
```

Test the frontend:

```
curl -H "Host: myapp.example.com" http://localhost:8080/
```

Test the API:

```
curl -H "Host: myapp.example.com" http://localhost:8080/api
```

**Step 5: Add Header-Based Routing**

Enhance the VirtualService to include header-based routing:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: myapp-external
  namespace: ${NAMESPACE}
spec:
  hosts:
  - myapp.example.com
  gateways:
  - my-gateway
  http:
  - match:
    - headers:
        x-canary:
          exact: "true"
    route:
    - destination:
        host: frontend
        subset: v2
      weight: 100
  - match:
    - uri:
        prefix: "/api"
    route:
    - destination:
        host: backend
        port:
          number: 80
      weight: 100
  - route:
    - destination:
        host: frontend
        port:
          number: 80
      weight: 100
EOF
```

Test with the canary header:

```
curl -H "Host: myapp.example.com" -H "x-canary: true" http://localhost:8080/
```

## Summary

In these exercises, you have:

- Created and configured Gateways to accept external traffic
- Created VirtualServices with basic routing patterns
- Implemented path-based routing to direct traffic based on URI paths
- Configured weighted traffic splitting for canary deployments
- Combined Gateways and VirtualServices for complete external traffic management
- Used DestinationRules to define service subsets for version-based routing

These skills form the foundation of Istio traffic management and are essential for implementing advanced deployment strategies in production environments.

## Troubleshooting

If you encounter issues:

- **Gateway not accepting traffic**: Verify the Gateway selector matches your ingress gateway pods and check that the gateway pods are running
- **VirtualService not routing**: Ensure the VirtualService hosts match the Gateway hosts, and verify the destination services exist
- **Traffic not splitting correctly**: Check that DestinationRule subsets match the pod labels, and verify the weight percentages sum to 100
- **External access not working**: Verify the ingress gateway service has an external IP or use port-forwarding for testing
- **Path matching not working**: Check the match conditions are correct and that routes are ordered properly (most specific first)

# Module 3: Advanced Traffic Management and Security

## Content

This module explores advanced features of DestinationRules and Istio's security capabilities. You will learn about load balancing, failover, and circuit breaking in DestinationRules, as well as how to implement request authentication and authorization policies to secure your services.

### DestinationRules: Beyond the Basics

You've already encountered DestinationRules in previous modules. Let's review what we've covered and then explore additional powerful features.

#### What We've Already Seen

In Module 1, you learned that DestinationRules can configure mTLS settings. In Module 2, you used DestinationRules to define service subsets for traffic splitting. These are fundamental uses of DestinationRules:

- **mTLS Configuration**: Setting TLS mode (e.g., `ISTIO_MUTUAL`) for secure service-to-service communication
- **Subsets**: Defining service versions or variants using labels for routing purposes

DestinationRules are versatile resources that define **how** traffic should be handled when it reaches a destination service. They complement VirtualServices, which define **where** traffic should go.

#### Load Balancing

One of the most important features of DestinationRules is configuring load balancing algorithms. By default, Istio uses round-robin load balancing, but you can configure various algorithms to suit your needs.

##### Load Balancing Algorithms

Istio supports several load balancing algorithms:

- **ROUND_ROBIN**: Distributes requests evenly across all endpoints (default)
- **LEAST_CONN**: Routes to the endpoint with the fewest active connections
- **RANDOM**: Randomly selects an endpoint
- **PASSTHROUGH**: Passes requests directly to the original destination without load balancing

##### Configuring Load Balancing

Here's an example of configuring load balancing in a DestinationRule:

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-service
spec:
  host: my-service
  trafficPolicy:
    loadBalancer:
      simple: LEAST_CONN
  subsets:
  - name: v1
    labels:
      version: v1
    trafficPolicy:
      loadBalancer:
        simple: ROUND_ROBIN
  - name: v2
    labels:
      version: v2
    trafficPolicy:
      loadBalancer:
        simple: RANDOM
```

This example shows:
* Global load balancing policy at the DestinationRule level
* Subset-specific load balancing policies that override the global policy

#### Locality-Aware Load Balancing

Istio also supports locality-aware load balancing, which prioritizes endpoints in the same zone or region:

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-service
spec:
  host: my-service
  trafficPolicy:
    loadBalancer:
      localityLbSetting:
        enabled: true
        distribute:
        - from: "region1/zone1/*"
          to:
            "region1/zone1/*": 80
            "region1/zone2/*": 20
```

This configuration ensures that traffic prefers local endpoints while allowing failover to other zones when needed.

### Failover

DestinationRules enable you to configure failover behavior, allowing automatic redirection to backup services when primary services fail.

#### Outlier Detection

Failover relies on outlier detection, which identifies unhealthy endpoints and removes them from the load balancing pool. Configure outlier detection like this:

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-service
spec:
  host: my-service
  trafficPolicy:
    outlierDetection:
      consecutiveErrors: 5
      interval: 30s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
      minHealthPercent: 50
```

Key parameters:

- **consecutiveErrors**: Number of consecutive errors before ejecting an endpoint
- **interval**: Time window for error counting
- **baseEjectionTime**: Minimum time an endpoint is ejected
- **maxEjectionPercent**: Maximum percentage of endpoints that can be ejected
- **minHealthPercent**: Minimum percentage of healthy endpoints required

#### Failover Configuration

You can configure failover to backup services using subsets:

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-service
spec:
  host: my-service
  trafficPolicy:
    outlierDetection:
      consecutiveErrors: 3
```

```
      interval: 30s
  subsets:
  - name: primary
    labels:
      version: v1
  - name: backup
    labels:
      version: v2
    trafficPolicy:
      outlierDetection:
        consecutiveErrors: 5
```

When combined with a VirtualService, you can route traffic to the backup subset when the primary fails.

### Circuit Breaking

Circuit breaking is a critical resilience pattern that prevents cascading failures by stopping requests to unhealthy services.

#### What is Circuit Breaking?

A circuit breaker monitors service health and, when failures exceed a threshold, "opens" the circuit to stop sending requests. This prevents overwhelming a failing service and allows it to recover.

#### Circuit Breaker Configuration

Configure circuit breaking in DestinationRules:

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: my-service
spec:
  host: my-service
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 100
      http:
        http1MaxPendingRequests: 10
        http2MaxRequests: 100
        maxRequestsPerConnection: 2
    outlierDetection:
      consecutiveErrors: 5
      interval: 30s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
```

Connection pool settings:

- **maxConnections**: Maximum number of TCP connections
- **http1MaxPendingRequests**: Maximum pending HTTP/1.1 requests
- **http2MaxRequests**: Maximum concurrent HTTP/2 requests
- **maxRequestsPerConnection**: Maximum requests per connection

When these limits are exceeded, the circuit breaker opens and requests are rejected, preventing overload.

#### Circuit Breaker States

A circuit breaker operates in three states:

- **Closed**: Normal operation, requests flow through
- **Open**: Circuit is open, requests are immediately rejected
- **Half-Open**: Testing if the service has recovered, allowing limited requests

The circuit breaker automatically transitions between these states based on the configured thresholds.

## Request Authentication

Istio provides powerful authentication mechanisms to verify the identity of clients making requests to your services.

### What is Request Authentication?

Request Authentication (RequestAuthentication) verifies JSON Web Tokens (JWTs) attached to requests. It validates that requests come from trusted sources and extracts identity information from tokens.

Request Authentication is different from PeerAuthentication:

- **PeerAuthentication**: Verifies service-to-service communication (mTLS)
- **RequestAuthentication**: Verifies end-user identity (JWT tokens)

### RequestAuthentication Resource

Create a RequestAuthentication resource to validate JWTs:

```
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: jwt-auth
  namespace: my-namespace
spec:
  selector:
```

```
    matchLabels:
      app: my-app
  jwtRules:
  - issuer: "https://auth.example.com"
    jwksUri: "https://auth.example.com/.well-known/jwks.json"
```

Key components:

- **selector**: Specifies which workloads this policy applies to
- **jwtRules**: Defines JWT validation rules
- **issuer**: The JWT issuer that must match the token
- **jwksUri**: URI to fetch JSON Web Key Set for token validation

#### JWT Validation

RequestAuthentication validates:

- Token signature using keys from the JWKS endpoint
- Token expiration (exp claim)
- Token issuer (iss claim)
- Token audience (aud claim), if specified

If validation fails, the request is rejected with a 401 Unauthorized response.

#### Multiple JWT Providers

You can configure multiple JWT providers for different authentication sources:

```
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: multi-jwt-auth
spec:
  selector:
    matchLabels:
      app: my-app
  jwtRules:
  - issuer: "https://auth1.example.com"
    jwksUri: "https://auth1.example.com/.well-known/jwks.json"
  - issuer: "https://auth2.example.com"
    jwksUri: "https://auth2.example.com/.well-known/jwks.json"
```

Istio will accept tokens from any of the configured issuers.

#### RequestAuthentication Best Practices

When implementing RequestAuthentication:

- Use HTTPS for JWKS endpoints to prevent man-in-the-middle attacks
- Configure appropriate token expiration times
- Use specific selectors to limit the scope of authentication policies
- Monitor authentication failures to detect potential attacks
- Combine with AuthorizationPolicy for complete access control

## Authorization

While RequestAuthentication verifies **who** is making a request, AuthorizationPolicy controls **what** they can do.

### What is Authorization?

Authorization determines whether an authenticated user or service is allowed to perform a specific action. Istio's AuthorizationPolicy provides fine-grained access control based on various request attributes.

### AuthorizationPolicy Resource

Create an AuthorizationPolicy to control access:

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-specific-users
  namespace: my-namespace
spec:
  selector:
    matchLabels:
      app: my-app
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["auth.example.com/*"]
    to:
    - operation:
        methods: ["GET", "POST"]
        paths: ["/api/*"]
```

Key components:

- **selector**: Specifies which workloads this policy applies to
- **action**: `ALLOW` or `DENY` (default is `ALLOW`)
- **rules**: Defines the conditions for the policy

### Authorization Actions

AuthorizationPolicy supports three actions:

- **ALLOW**: Explicitly allows matching requests
- **DENY**: Explicitly denies matching requests
- **AUDIT**: Logs matching requests without allowing or denying (for monitoring)

### Authorization Rules

Authorization rules can match on various request attributes:

#### Source-Based Rules

Match requests based on their source:

```
rules:
- from:
  - source:
      principals: ["cluster.local/ns/default/sa/my-service"]
      namespaces: ["default"]
      requestPrincipals: ["auth.example.com/user123"]
```

- **principals**: Service account identities (from mTLS)
- **namespaces**: Source namespaces
- **requestPrincipals**: End-user identities (from JWT)

#### Operation-Based Rules

Match requests based on the operation:

```
rules:
- to:
  - operation:
      methods: ["GET", "POST"]
      paths: ["/api/*"]
      hosts: ["api.example.com"]
```

- **methods**: HTTP methods (GET, POST, PUT, DELETE, etc.)
- **paths**: URI paths (supports exact, prefix, suffix, regex)
- **hosts**: Host headers

#### Custom Attribute Rules

Match requests based on custom attributes:

```
rules:
- when:
  - key: request.headers[user-agent]
    values: ["Mozilla/*"]
  - key: source.ip
    values: ["192.168.1.0/24"]
```

## Deny Rules

Create explicit deny rules to block specific requests:

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-admin
spec:
  selector:
    matchLabels:
      app: my-app
  action: DENY
  rules:
  - to:
    - operation:
        paths: ["/admin/*"]
```

This policy denies all requests to `/admin/*` paths.

#### Combining Multiple Policies

You can create multiple AuthorizationPolicies for different scenarios:

```
# Policy 1: Allow authenticated users to access API
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-api
spec:
  selector:
    matchLabels:
      app: my-app
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["*"]
    to:
    - operation:
```

```
        paths: ["/api/*"]

---
# Policy 2: Deny access to admin endpoints
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-admin
spec:
  selector:
    matchLabels:
      app: my-app
  action: DENY
  rules:
  - to:
    - operation:
        paths: ["/admin/*"]
```

Policies are evaluated in order, and DENY policies take precedence over ALLOW policies.

**Authorization Best Practices**

When implementing authorization:

- Follow the principle of least privilege
- Use specific selectors to limit policy scope
- Combine RequestAuthentication with AuthorizationPolicy for complete security
- Test authorization policies thoroughly before deploying to production
- Monitor authorization denials to detect potential security issues
- Document policies clearly for team understanding
- Use namespaces to isolate authorization policies when appropriate

## Combining Authentication and Authorization

For complete security, combine RequestAuthentication and AuthorizationPolicy:

1. **RequestAuthentication**: Verifies the identity of the requester (via JWT)
2. **AuthorizationPolicy**: Controls what the authenticated requester can do

Example workflow:

```
# Step 1: Require JWT authentication
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: require-jwt
spec:
```

```
  selector:
    matchLabels:
      app: my-app
  jwtRules:
  - issuer: "https://auth.example.com"
    jwksUri: "https://auth.example.com/.well-known/jwks.json"

---
# Step 2: Authorize based on identity
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: require-auth
spec:
  selector:
    matchLabels:
      app: my-app
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["auth.example.com/*"]
    to:
    - operation:
        methods: ["GET", "POST"]
```

This configuration:
* Requires valid JWTs from the specified issuer
* Only allows GET and POST requests from authenticated users
* Rejects all other requests

## Summary

In this module, you have learned:

- DestinationRules provide advanced traffic management beyond mTLS and subsets
- Load balancing can be configured with various algorithms (round-robin, least-conn, random)
- Failover and outlier detection help maintain service availability
- Circuit breaking prevents cascading failures by limiting connections and requests
- RequestAuthentication validates JWT tokens to verify user identity
- AuthorizationPolicy provides fine-grained access control based on various attributes
- Authentication and authorization work together to provide complete security

These advanced features enable you to build resilient, secure microservices architectures with Istio.

### Next Steps

In the next module, you will explore observability features that help you monitor, trace, and understand the behavior of your service mesh.

# Exercises

This exercise section provides hands-on practice with advanced DestinationRule features and Istio security. You will learn how to configure load balancing, implement failover and circuit breaking, set up request authentication, and create authorization policies.

## Prerequisites

Before starting these exercises, ensure you have:

- Access to an OpenShift cluster with Service Mesh 3 (Istio >= 1.23) installed
- Your namespace (`$NAMESPACE`) configured with sidecar injection enabled
- The `oc` CLI tool installed and configured
- Basic understanding of DestinationRules, VirtualServices, and Kubernetes Services

## Exercise 1: Configure Load Balancing in DestinationRules

In this exercise, you will configure different load balancing algorithms in DestinationRules.

#### Step 1: Set Your Namespace

Ensure your namespace variable is set:

```
export NAMESPACE={USER}-sandbox
```

#### Step 2: Deploy a Multi-Pod Service

Deploy a service with multiple replicas to observe load balancing:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: loadtest-service
  namespace: ${NAMESPACE}
spec:
  replicas: 3
  selector:
    matchLabels:
      app: loadtest
      version: v1
  template:
```

```
    metadata:
      labels:
        app: loadtest
        version: v1
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
        env:
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
---
apiVersion: v1
kind: Service
metadata:
  name: loadtest-service
  namespace: ${NAMESPACE}
spec:
  selector:
    app: loadtest
  ports:
  - port: 80
    targetPort: 80
    name: http
EOF
```

Wait for all pods to be ready:

```
oc get pods -n $NAMESPACE -l app=loadtest
```

**Step 3: Create DestinationRule with Round-Robin Load Balancing**

Create a DestinationRule with round-robin load balancing (default):

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: loadtest-service
  namespace: ${NAMESPACE}
spec:
  host: loadtest-service
  trafficPolicy:
    loadBalancer:
      simple: ROUND_ROBIN
```

```
EOF
```

Verify the DestinationRule:

```
oc get destinationrule loadtest-service -n $NAMESPACE -o yaml
```

**Step 4: Test Round-Robin Load Balancing**

Send multiple requests and observe the distribution. Check the logs to see which pod handles each request:

```
for i in {1..9}; do
  oc run curl-test-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://loadtest-service > /dev/null
done
```

Check the access logs to see the distribution:

```
oc logs -n $NAMESPACE -l app=loadtest -c istio-proxy --tail=20 | grep -i "GET"
```

**Step 5: Change to Least-Connection Load Balancing**

Update the DestinationRule to use least-connection load balancing:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: loadtest-service
  namespace: ${NAMESPACE}
spec:
  host: loadtest-service
  trafficPolicy:
    loadBalancer:
      simple: LEAST_CONN
EOF
```

Test the least-connection algorithm:

```
for i in {1..9}; do
  oc run curl-test-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://loadtest-service > /dev/null
done
```

#### Step 6: Configure Random Load Balancing

Update to random load balancing:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: loadtest-service
  namespace: ${NAMESPACE}
spec:
  host: loadtest-service
  trafficPolicy:
    loadBalancer:
      simple: RANDOM
EOF
```

## Exercise 2: Implement Failover and Outlier Detection

In this exercise, you will configure outlier detection and failover behavior.

#### Step 1: Deploy Primary and Backup Services

Deploy a primary service and a backup service:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: primary-service
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: primary
      version: v1
  template:
    metadata:
      labels:
        app: primary
        version: v1
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: primary-service
  namespace: ${NAMESPACE}
spec:
  selector:
    app: primary
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backup-service
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backup
      version: v1
  template:
    metadata:
      labels:
        app: backup
        version: v1
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: backup-service
  namespace: ${NAMESPACE}
spec:
  selector:
    app: backup
  ports:
  - port: 80
    targetPort: 80
EOF
```

**Step 2: Create DestinationRule with Outlier Detection**

Create a DestinationRule with outlier detection configured:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: primary-service
  namespace: ${NAMESPACE}
spec:
  host: primary-service
  trafficPolicy:
    outlierDetection:
      consecutiveErrors: 3
      interval: 30s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
      minHealthPercent: 50
  subsets:
  - name: primary
    labels:
      version: v1
  - name: backup
    labels:
      version: v1
EOF
```

**Step 3: Create VirtualService for Failover**

Create a VirtualService that routes to the backup when primary fails:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: primary-service
  namespace: ${NAMESPACE}
spec:
  hosts:
  - primary-service
  http:
  - route:
    - destination:
        host: primary-service
        subset: primary
      weight: 100
    fault:
      delay:
```

```
        percentage:
          value: 50
        fixedDelay: 5s
    retries:
      attempts: 3
      perTryTimeout: 2s
EOF
```

The fault injection in this example simulates failures. In production, you would remove the fault section.

**Step 4: Verify Outlier Detection Configuration**

Check the DestinationRule configuration:

```
oc get destinationrule primary-service -n $NAMESPACE -o yaml
```

**Step 5: Test Failover Behavior**

Send requests and observe the behavior:

```
for i in {1..10}; do
  oc run curl-test-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -v http://primary-service --max-time 10
done
```

Check the logs to see outlier detection in action:

```
oc logs -n $NAMESPACE -l app=primary -c istio-proxy --tail=30
```

## Exercise 3: Implement Circuit Breaking

In this exercise, you will configure circuit breaking to prevent cascading failures.

**Step 1: Deploy a Service for Circuit Breaking**

Deploy a service that we'll use to test circuit breaking:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: circuit-service
  namespace: ${NAMESPACE}
spec:
  replicas: 2
```

```
  selector:
    matchLabels:
      app: circuit
  template:
    metadata:
      labels:
        app: circuit
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: circuit-service
  namespace: ${NAMESPACE}
spec:
  selector:
    app: circuit
  ports:
  - port: 80
    targetPort: 80
EOF
```

**Step 2: Create DestinationRule with Circuit Breaking**

Create a DestinationRule with circuit breaking configuration:

```
oc apply -f - <<EOF
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: circuit-service
  namespace: ${NAMESPACE}
spec:
  host: circuit-service
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 2
      http:
        http1MaxPendingRequests: 2
        http2MaxRequests: 2
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutiveErrors: 2
      interval: 30s
```

```
      baseEjectionTime: 30s
      maxEjectionPercent: 100
EOF
```

**Step 3: Verify Circuit Breaking Configuration**

Check the DestinationRule:

```
oc get destinationrule circuit-service -n $NAMESPACE -o yaml
```

**Step 4: Test Circuit Breaking**

Deploy a client that will generate load to test circuit breaking:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: circuit-client
  namespace: ${NAMESPACE}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: circuit-client
  template:
    metadata:
      labels:
        app: circuit-client
    spec:
      containers:
      - name: curl
        image: curlimages/curl:latest
        command: ["sleep", "3600"]
EOF
```

Send multiple concurrent requests to trigger circuit breaking:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=circuit-client -o
jsonpath='{.items[0].metadata.name}')
oc exec -n $NAMESPACE $POD_NAME -- sh -c 'for i in $(seq 1 10); do curl -s
http://circuit-service & done; wait'
```

Check the logs to see circuit breaking behavior:

```
oc logs -n $NAMESPACE -l app=circuit-client -c istio-proxy --tail=20
```

```
oc logs -n $NAMESPACE -l app=circuit -c istio-proxy --tail=20
```

## Exercise 4: Configure Request Authentication with JWT

In this exercise, you will set up RequestAuthentication to validate JWT tokens.

### Step 1: Deploy a Test Application

Deploy an application that will require authentication:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: auth-service
  template:
    metadata:
      labels:
        app: auth-service
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: auth-service
  namespace: ${NAMESPACE}
spec:
  selector:
    app: auth-service
  ports:
  - port: 80
    targetPort: 80
EOF
```

### Step 2: Create a Simple JWT for Testing

For this exercise, we'll use a simple JWT issuer. In production, you would use a proper authentication service. Create a RequestAuthentication policy:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: jwt-auth
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: auth-service
  jwtRules:
  - issuer: "testing@secure.istio.io"
    jwksUri: "https://raw.githubusercontent.com/istio/istio/release-
1.23/security/tools/jwt/samples/jwks.json"
EOF
```

This uses Istio's sample JWT for testing. In production, use your own JWT issuer and JWKS endpoint.

**Step 3: Verify RequestAuthentication**

Check that the RequestAuthentication was created:

```
oc get requestauthentication -n $NAMESPACE
oc get requestauthentication jwt-auth -n $NAMESPACE -o yaml
```

**Step 4: Test Without JWT Token**

Try to access the service without a JWT token:

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v http://auth-service
```

The request should succeed because RequestAuthentication only validates tokens if they are present. To require tokens, you need AuthorizationPolicy (covered in the next exercise).

**Step 5: Test With Invalid JWT Token**

Test with an invalid token:

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -H "Authorization: Bearer invalid-token" http://auth-service
```

#### Step 6: Test With Valid JWT Token

Get a sample JWT token from Istio's test tokens:

```
TOKEN=$(curl -s https://raw.githubusercontent.com/istio/istio/release-
1.23/security/tools/jwt/samples/demo.jwt)
echo $TOKEN
```

Test with a valid token:

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -H "Authorization: Bearer $TOKEN" http://auth-service
```

### Exercise 5: Create Authorization Policies

In this exercise, you will create AuthorizationPolicies to control access to your services.

#### Step 1: Create an AuthorizationPolicy to Require Authentication

Create an AuthorizationPolicy that requires a valid JWT token:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: require-jwt
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: auth-service
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["*"]
EOF
```

This policy allows requests from any authenticated principal (any valid JWT).

#### Step 2: Test Authorization Policy

Try accessing without a token (should be denied):

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v http://auth-service
```

Try with a valid token (should be allowed):

```
TOKEN=$(curl -s https://raw.githubusercontent.com/istio/istio/release-
1.23/security/tools/jwt/samples/demo.jwt)
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -H "Authorization: Bearer $TOKEN" http://auth-service
```

**Step 3: Create Method-Based Authorization**

Create an AuthorizationPolicy that only allows GET and POST methods:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-methods
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: auth-service
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["*"]
    to:
    - operation:
        methods: ["GET", "POST"]
EOF
```

**Step 4: Test Method-Based Authorization**

Test with GET (should work):

```
TOKEN=$(curl -s https://raw.githubusercontent.com/istio/istio/release-
1.23/security/tools/jwt/samples/demo.jwt)
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -X GET -H "Authorization: Bearer $TOKEN" http://auth-service
```

Test with DELETE (should be denied):

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -X DELETE -H "Authorization: Bearer $TOKEN" http://auth-service
```

**Step 5: Create Path-Based Authorization**

Create an AuthorizationPolicy that restricts access to specific paths:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-paths
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: auth-service
  action: ALLOW
  rules:
  - from:
    - source:
        requestPrincipals: ["*"]
    to:
    - operation:
        paths: ["/api/*", "/public/*"]
EOF
```

**Step 6: Create a Deny Policy**

Create an explicit deny policy for admin endpoints:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-admin
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: auth-service
  action: DENY
  rules:
  - to:
    - operation:
        paths: ["/admin/*"]
EOF
```

Test the deny policy:

```
TOKEN=$(curl -s https://raw.githubusercontent.com/istio/istio/release-
```

```
1.23/security/tools/jwt/samples/demo.jwt)
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -H "Authorization: Bearer $TOKEN" http://auth-service/admin/test
```

**Step 7: Verify All Authorization Policies**

List all AuthorizationPolicies:

```
oc get authorizationpolicy -n $NAMESPACE
```

View the configurations:

```
oc get authorizationpolicy -n $NAMESPACE -o yaml
```

## Exercise 6: Combine Authentication and Authorization

In this exercise, you will combine RequestAuthentication and AuthorizationPolicy for complete security.

**Step 1: Deploy a New Service**

Deploy a service for the combined authentication and authorization exercise:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: secure-service
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
      app: secure-service
  template:
    metadata:
      labels:
        app: secure-service
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: secure-service
  namespace: ${NAMESPACE}
spec:
  selector:
    app: secure-service
  ports:
  - port: 80
    targetPort: 80
EOF
```

**Step 2: Create RequestAuthentication**

Create a RequestAuthentication policy for the service:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: secure-jwt-auth
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: secure-service
  jwtRules:
  - issuer: "testing@secure.istio.io"
    jwksUri: "https://raw.githubusercontent.com/istio/istio/release-
1.23/security/tools/jwt/samples/jwks.json"
EOF
```

**Step 3: Create Comprehensive AuthorizationPolicy**

Create an AuthorizationPolicy that combines multiple rules:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: secure-policy
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: secure-service
  action: ALLOW
  rules:
  - from:
    - source:
```

```
        requestPrincipals: ["testing@secure.istio.io/*"]
    to:
    - operation:
        methods: ["GET", "POST"]
        paths: ["/api/*"]
  - from:
    - source:
        requestPrincipals: ["testing@secure.istio.io/*"]
    to:
    - operation:
        methods: ["GET"]
        paths: ["/public/*"]
EOF
```

**Step 4: Create Deny Policy for Admin**

Create a deny policy for admin endpoints:

```
oc apply -f - <<EOF
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-secure-admin
  namespace: ${NAMESPACE}
spec:
  selector:
    matchLabels:
      app: secure-service
  action: DENY
  rules:
  - to:
    - operation:
        paths: ["/admin/*"]
EOF
```

**Step 5: Test Complete Security Configuration**

Test without token (should be denied):

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v http://secure-service/api/test
```

Test with valid token on allowed path (should work):

```
TOKEN=$(curl -s https://raw.githubusercontent.com/istio/istio/release-
1.23/security/tools/jwt/samples/demo.jwt)
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
```

```
-- curl -v -H "Authorization: Bearer $TOKEN" http://secure-service/api/test
```

Test with valid token on denied path (should be denied):

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -H "Authorization: Bearer $TOKEN" http://secure-service/admin/test
```

Test with valid token on public path (should work):

```
oc run curl-test --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -v -H "Authorization: Bearer $TOKEN" http://secure-service/public/test
```

**Step 6: Verify Complete Configuration**

View all security policies:

```
oc get requestauthentication,authorizationpolicy -n $NAMESPACE
```

View detailed configurations:

```
oc get requestauthentication secure-jwt-auth -n $NAMESPACE -o yaml
oc get authorizationpolicy secure-policy -n $NAMESPACE -o yaml
oc get authorizationpolicy deny-secure-admin -n $NAMESPACE -o yaml
```

## Summary

In these exercises, you have:

- Configured different load balancing algorithms (round-robin, least-connection, random) in DestinationRules
- Implemented outlier detection and failover behavior
- Configured circuit breaking to prevent cascading failures
- Set up RequestAuthentication to validate JWT tokens
- Created AuthorizationPolicies with various rules (method-based, path-based, deny policies)
- Combined authentication and authorization for complete security

These advanced features enable you to build resilient, secure microservices with fine-grained access control.

## Troubleshooting

If you encounter issues:

- **Load balancing not working**: Verify the DestinationRule is applied and check that multiple endpoints are available
- **Failover not triggering**: Check outlier detection settings and ensure errors are being generated
- **Circuit breaker not opening**: Verify connection pool limits are appropriate and that load is sufficient
- **JWT authentication failing**: Verify the JWKS endpoint is accessible and the token format is correct
- **Authorization denying valid requests**: Check the request principal matches the policy rules and verify policy order (DENY takes precedence)
- **Policies not applying**: Verify selector labels match the workload labels and check namespace scope

# Module 4: Observability

## Content

This module explores how Istio provides comprehensive observability into your microservices architecture. You will learn about the three pillars of observability—metrics, traces, and logs—how Istio generates these signals in standard formats, and how to explore them using local tools or integrate them with your APM (Application Performance Monitoring) solution.

### The Three Pillars of Observability

Observability is the ability to understand the internal state of a system by examining its outputs. In microservices architectures, observability is crucial because the distributed nature of services makes it difficult to understand system behavior without proper instrumentation.

Istio provides three fundamental signals that form the foundation of observability:

- **Metrics**: Quantitative measurements of system behavior over time
- **Traces**: End-to-end request flows across service boundaries
- **Logs**: Detailed records of events and activities

Together, these three signals provide a complete picture of your microservices' health, performance, and behavior.

### Metrics: Quantitative System Insights

Metrics are numerical measurements collected over time that provide insights into system performance, health, and behavior. Istio automatically generates rich metrics for all service-to-service communication without requiring any changes to your application code.

#### What Metrics Does Istio Provide?

Istio generates metrics at multiple levels:

- **Service Metrics**: Request rates, error rates, and latency for each service
- **Workload Metrics**: Metrics specific to individual workload instances (pods)
- **Proxy Metrics**: Detailed Envoy proxy metrics including connection counts, retries, and circuit breaker states
- **Control Plane Metrics**: Metrics about the Istio control plane itself

#### Standard Metrics Format

Istio exposes metrics in the Prometheus format, which has become the de facto standard for metrics in cloud-native environments. Prometheus metrics are:

- **Text-based**: Human-readable format

- **Pull-based**: Prometheus scrapes metrics from endpoints
- **Dimensional**: Metrics include labels for filtering and aggregation
- **Time-series**: Optimized for time-series databases

#### Key Istio Metrics

Some of the most important metrics Istio provides include:

- `istio_requests_total`: Total number of requests
- `istio_request_duration_milliseconds`: Request latency
- `istio_request_bytes`: Request payload size
- `istio_response_bytes`: Response payload size
- `istio_tcp_connections_opened_total`: TCP connection statistics
- `istio_tcp_connections_closed_total`: TCP connection statistics

Each metric includes labels such as:
* `source_service`: The service making the request
* `destination_service`: The service receiving the request
* `response_code`: HTTP response code
* `request_protocol`: Protocol used (HTTP, gRPC, etc.)

#### Accessing Metrics Locally

For local exploration, you can use tools like:

- **Kiali**: Provides a service mesh dashboard with built-in metrics visualization
- **Prometheus**: Direct access to metrics endpoints for querying and alerting
- **Grafana**: Create custom dashboards using Prometheus as a data source

These tools allow you to explore metrics in real-time, create dashboards, and set up alerts based on metric thresholds.

#### Integrating with APM Solutions

Istio metrics can be integrated with remote APM (Application Performance Monitoring) solutions such as:

- **Datadog**: Cloud-based monitoring and analytics platform
- **New Relic**: Application performance monitoring and observability platform
- **Dynatrace**: Full-stack observability platform
- **Splunk**: Enterprise observability and security platform

These solutions typically provide:
* Advanced analytics and machine learning capabilities
* Long-term metric storage and retention

* Cross-platform correlation
* Enterprise-grade alerting and incident management
* Custom dashboards and reporting

## Traces: End-to-End Request Visibility

Distributed tracing provides visibility into how requests flow through your microservices architecture. It shows the complete path a request takes from entry point to final destination, including all intermediate services.

### What is Distributed Tracing?

Distributed tracing tracks requests as they traverse multiple services. Each request is assigned a unique trace ID, and each service interaction creates a span. Spans are connected to form a trace that shows the complete request journey.

A trace consists of:
* **Trace ID**: Unique identifier for the entire request
* **Spans**: Individual operations within the trace
* **Parent-Child Relationships**: How spans relate to each other
* **Timing Information**: Duration of each operation
* **Metadata**: Additional context about each operation

### How Istio Enables Tracing

Istio automatically instruments all service-to-service communication to generate traces. The Envoy sidecar proxies:

- Generate spans for each request
- Propagate trace context between services
- Add metadata about the request (HTTP headers, response codes, etc.)
- Measure latency at each hop

This happens transparently—your applications don't need to be modified to participate in distributed tracing.

### Standard Tracing Formats

Istio supports multiple tracing formats:

- **OpenTelemetry**: Industry-standard observability framework
- **Zipkin**: Open-source distributed tracing system
- **Jaeger**: Open-source end-to-end distributed tracing
- **Lightstep**: Commercial distributed tracing platform

These formats are interoperable, allowing you to use different tools at different stages of your observability journey.

### Accessing Traces Locally

For local exploration, you can use:

- **Kiali**: Provides trace visualization integrated with service topology
- **Tempo**: Grafana's open-source distributed tracing backend
- **Jaeger**: Popular open-source tracing UI
- **Zipkin**: Simple and fast distributed tracing system

These tools allow you to:
* Search traces by service, operation, or tag
* View trace timelines and identify bottlenecks
* Analyze error rates and latency patterns
* Understand service dependencies

### Integrating with APM Solutions

Remote APM solutions provide advanced trace analysis:

- **Correlation**: Correlate traces with metrics and logs
- **Service Maps**: Automatic generation of service dependency graphs
- **Anomaly Detection**: Identify unusual patterns in trace data
- **Root Cause Analysis**: AI-powered analysis to identify issues
- **Long-term Storage**: Retain traces for historical analysis

## Logs: Detailed Event Records

Logs provide detailed, timestamped records of events and activities within your system. While metrics give you the "what" and traces give you the "where," logs provide the "why" and "how."

### What Logs Does Istio Generate?

Istio generates comprehensive logs through the Envoy sidecar proxies:

- **Access Logs**: Every HTTP/gRPC request and response
- **Error Logs**: Detailed error information
- **Debug Logs**: Low-level debugging information
- **Audit Logs**: Security and policy enforcement events

Access logs are particularly valuable because they capture:
* Request and response headers
* Request and response body (configurable)
* Response codes and status
* Timing information
* Connection metadata

#### Standard Log Formats

Istio logs are generated in standard formats:

- **JSON**: Structured JSON format for easy parsing
- **Text**: Human-readable text format
- **OpenTelemetry Logs**: OTLP format for integration with observability platforms

The structured nature of these logs makes them easy to:
* Parse and analyze programmatically
* Filter and search
* Correlate with metrics and traces
* Integrate with log aggregation systems

#### Accessing Logs Locally

For local exploration, you can:

- **View logs directly**: Using `kubectl logs` or `oc logs` commands
- **Use Kiali**: View logs in the context of service interactions
- **Use Tempo**: Correlate logs with traces
- **Use log aggregation tools**: Like Loki (Grafana's log aggregation system)

These tools help you:
* Search logs by service, time range, or content
* Filter logs by severity or type
* Correlate logs with specific requests or traces
* Identify patterns and anomalies

#### Integrating with APM Solutions

Remote APM and log management solutions provide:

- **Centralized Log Aggregation**: Collect logs from all services
- **Advanced Search**: Full-text search across all logs
- **Log Analytics**: Machine learning-powered log analysis
- **Alerting**: Alert on log patterns and errors
- **Compliance**: Long-term storage for compliance requirements

### Standard Formats: The Key to Flexibility

One of Istio's greatest strengths is its use of standard, open formats for all observability signals. This standardization provides several benefits:

#### Vendor Independence

By using standard formats, you're not locked into a specific vendor or tool. You can:

- Start with local tools (Kiali, Tempo) for development
- Migrate to enterprise APM solutions for production
- Use multiple tools simultaneously
- Switch tools without changing your application code

**Tool Ecosystem**

Standard formats mean you can leverage the entire ecosystem of observability tools:

- **Open-source tools**: Prometheus, Grafana, Jaeger, Tempo, Kiali
- **Commercial APM solutions**: Datadog, New Relic, Dynatrace, Splunk
- **Cloud-native tools**: AWS CloudWatch, Google Cloud Operations, Azure Monitor
- **Custom solutions**: Build your own dashboards and analytics

**Interoperability**

Standard formats enable interoperability between tools:

- Metrics from Istio can feed into any Prometheus-compatible system
- Traces can be exported to any OpenTelemetry-compatible backend
- Logs can be consumed by any log aggregation system
- You can mix and match tools based on your needs

**OpenTelemetry: The Universal Observability Standard**

OpenTelemetry (OTel) is an open-source observability framework that provides a unified standard for collecting telemetry data. It's becoming the industry standard for observability instrumentation and is increasingly supported by Istio and the broader observability ecosystem.

OpenTelemetry provides:

- **Unified APIs**: Standard APIs for metrics, traces, and logs across multiple languages
- **Vendor-Neutral**: Not tied to any specific vendor or backend
- **Instrumentation Libraries**: Pre-built instrumentation for common frameworks and libraries
- **Multiple Export Formats**: Can export to Prometheus, Jaeger, Zipkin, and many other backends
- **Future-Proof**: Actively developed and widely adopted across the industry

Istio's support for OpenTelemetry means:

- Traces can be exported in OTLP (OpenTelemetry Protocol) format
- Metrics can be collected using OpenTelemetry collectors
- Logs can be structured using OpenTelemetry log formats
- Easy integration with OpenTelemetry-compatible APM solutions
- Ability to correlate data across different observability signals

By leveraging OpenTelemetry, you gain maximum flexibility to choose your observability tools while ensuring your instrumentation remains portable and future-proof.

## Local Exploration: Kiali and Tempo

For development and local environments, Kiali and Tempo provide powerful, integrated observability capabilities.

### Kiali: Service Mesh Observability

Kiali is a web-based console for Istio service mesh observability. It provides:

- **Service Graph**: Visual representation of service dependencies
- **Metrics Dashboard**: Pre-built dashboards for key metrics
- **Trace Visualization**: View and analyze distributed traces
- **Health Overview**: Service health status at a glance
- **Configuration Validation**: Validate Istio configuration

Kiali is particularly valuable because it:
* Understands Istio-specific concepts
* Provides context-aware visualizations
* Integrates metrics, traces, and configuration
* Requires no additional configuration

### Tempo: Distributed Tracing Backend

Tempo is Grafana's open-source distributed tracing backend. It provides:

- **High-performance trace storage**: Efficient storage of trace data
- **Grafana Integration**: Native integration with Grafana dashboards
- **Trace Correlation**: Correlate traces with metrics and logs
- **Simple Architecture**: Easy to deploy and operate

Tempo is ideal for:
* Organizations already using Grafana
* Teams wanting open-source solutions
* Environments requiring cost-effective trace storage
* Integration with existing observability stacks

## Remote APM Integration

For production environments, enterprise APM solutions provide advanced capabilities beyond what local tools offer.

### Why Use Remote APM?

Remote APM solutions offer:

- **Scalability**: Handle large-scale deployments
- **Advanced Analytics**: Machine learning and AI-powered insights
- **Enterprise Features**: SSO, RBAC, compliance, and governance
- **Long-term Storage**: Retain data for extended periods
- **Cross-platform Correlation**: Correlate data across multiple systems
- **Professional Support**: Vendor support and SLAs

#### Integration Patterns

Istio observability data can be integrated with APM solutions through:

- **Metrics Exporters**: Export Prometheus metrics to APM platforms
- **Trace Exporters**: Send traces via OpenTelemetry or native protocols
- **Log Forwarders**: Forward logs using standard protocols
- **API Integration**: Use APM APIs to push or pull data

#### Choosing the Right APM

When selecting an APM solution, consider:

- **Data Volume**: Can it handle your scale?
- **Retention**: How long can you retain data?
- **Cost**: Pricing model and total cost of ownership
- **Features**: Does it provide the analytics you need?
- **Integration**: How easily does it integrate with Istio?
- **Vendor Lock-in**: Can you migrate if needed?

## Putting Microservices Under a Microscope

The combination of metrics, traces, and logs from Istio provides unprecedented visibility into your microservices architecture. This observability enables you to:

#### Understand System Behavior

With comprehensive observability, you can:

- **Identify Bottlenecks**: See exactly where requests are slow
- **Understand Dependencies**: Visualize how services interact
- **Detect Anomalies**: Identify unusual patterns and behaviors
- **Track Changes**: Understand the impact of deployments

#### Debug Issues Quickly

When problems occur, observability helps you:

- **Trace Root Causes**: Follow requests through the system to find issues
- **Correlate Events**: Connect metrics, traces, and logs to understand failures
- **Isolate Problems**: Quickly identify which service is causing issues
- **Validate Fixes**: Confirm that changes resolve problems

#### Optimize Performance

Observability data enables optimization:

- **Identify Slow Operations**: Find operations that need optimization
- **Understand Resource Usage**: See which services consume the most resources
- **Validate Improvements**: Measure the impact of optimizations
- **Plan Capacity**: Use historical data to plan for growth

#### Ensure Reliability

Observability supports reliability:

- **Monitor SLOs**: Track service level objectives
- **Detect Degradation**: Identify issues before they become outages
- **Validate Resilience**: Confirm that circuit breakers and retries work
- **Audit Security**: Review access patterns and security events

## Best Practices for Observability

To get the most value from Istio's observability features:

#### Start with the Basics

- Begin with Kiali for a quick overview
- Enable access logging for key services
- Set up basic metrics dashboards
- Configure trace sampling appropriately

#### Use Standard Formats

- Stick with Prometheus for metrics
- Use OpenTelemetry for traces
- Use structured logging (JSON)
- This ensures future flexibility

#### Sample Appropriately

- Don't trace every request (too expensive)

- Use sampling rates (e.g., 1% or 10%)
- Increase sampling for error cases
- Adjust based on traffic volume

**Correlate Signals**

- Use trace IDs to correlate logs and traces
- Link metrics to specific services and operations
- Create dashboards that combine metrics, traces, and logs
- Use consistent labeling across all signals

**Monitor What Matters**

- Focus on business-critical metrics
- Set up alerts for important thresholds
- Track user-facing metrics (latency, errors)
- Monitor resource utilization

## Summary

In this module, you have learned:

- Istio provides three fundamental observability signals: metrics, traces, and logs
- These signals are generated automatically without application code changes
- All signals use standard formats (Prometheus, OpenTelemetry, structured logs)
- Local tools like Kiali and Tempo provide powerful exploration capabilities
- Remote APM solutions offer enterprise-grade observability for production
- Standard formats provide flexibility and vendor independence
- Comprehensive observability enables you to understand, debug, optimize, and secure your microservices

Observability is not just about collecting data—it's about gaining insights that enable you to build, operate, and improve your microservices architecture with confidence.

## Conclusion

Congratulations! You have completed the Istio Service Mesh Workshop. Throughout these four modules, you have learned:

- **Module 1**: The fundamentals of Istio, Envoy, and sidecar architecture
- **Module 2**: Traffic management with Gateways and VirtualServices
- **Module 3**: Advanced traffic management and security with DestinationRules, authentication, and authorization

- **Module 4**: Observability with metrics, traces, and logs

You now have the knowledge and skills to deploy, configure, secure, and observe Istio service mesh in your Kubernetes environment. Continue practicing with the exercises, explore the advanced features, and leverage Istio's capabilities to build resilient, secure, and observable microservices architectures.

# Exercises

This exercise section provides hands-on practice with accessing metrics and logs directly from Envoy sidecar proxies. You will learn how to query Envoy metrics, view access logs, and use built-in Istio tools to explore observability data without installing external tools.

## Prerequisites

Before starting these exercises, ensure you have:

- Access to an OpenShift cluster with Service Mesh 3 (Istio >= 1.23) installed
- Your namespace (`$NAMESPACE`) configured with sidecar injection enabled
- The `oc` CLI tool installed and configured
- Services deployed in your namespace (from previous modules)

## Exercise 1: Access Envoy Metrics

In this exercise, you will learn how to access Prometheus-formatted metrics directly from Envoy sidecar proxies.

### Step 1: Set Your Namespace

Ensure your namespace variable is set:

```
export NAMESPACE={USER}-sandbox
```

### Step 2: Deploy a Test Service

Deploy a simple service to generate metrics:

```
oc apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: metrics-test
  namespace: ${NAMESPACE}
spec:
  replicas: 2
  selector:
    matchLabels:
```

```
      app: metrics-test
  template:
    metadata:
      labels:
        app: metrics-test
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: metrics-test
  namespace: ${NAMESPACE}
spec:
  selector:
    app: metrics-test
  ports:
  - port: 80
    targetPort: 80
EOF
```

Wait for pods to be ready:

```
oc get pods -n $NAMESPACE -l app=metrics-test
```

**Step 3: Generate Some Traffic**

Generate traffic to create metrics:

```
for i in {1..20}; do
  oc run curl-test-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://metrics-test > /dev/null
done
```

**Step 4: Access Envoy Metrics Endpoint**

Envoy sidecars expose metrics on port 15020. Port-forward to access the metrics endpoint:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
oc port-forward -n $NAMESPACE $POD_NAME 15020:15020 &
```

Access the metrics endpoint:

```
curl http://localhost:15020/stats/prometheus
```

You should see Prometheus-formatted metrics. Look for metrics like:
* `istio_requests_total`
* `istio_request_duration_milliseconds`
* `istio_request_bytes`
* `istio_response_bytes`

**Step 5: Query Specific Metrics**

Query for specific metrics using grep:

```
curl -s http://localhost:15020/stats/prometheus | grep istio_requests_total
```

Query for request duration metrics:

```
curl -s http://localhost:15020/stats/prometheus | grep istio_request_duration
```

**Step 6: View All Envoy Stats**

View all Envoy statistics (not just Prometheus format):

```
curl http://localhost:15020/stats
```

This shows all available Envoy statistics in a different format.

**Step 7: Stop Port-Forwarding**

Stop the port-forward:

```
pkill -f "port-forward.*15020"
```

## Exercise 2: Query Metrics from Multiple Pods

In this exercise, you will query metrics from multiple Envoy sidecars and compare them.

**Step 1: Get All Pod Names**

Get the names of all pods for the service:

```
oc get pods -n $NAMESPACE -l app=metrics-test -o jsonpath='{range
.items[*]}{.metadata.name}{"\n"}{end}'
```

#### Step 2: Query Metrics from Each Pod

Query metrics from the first pod:

```
POD1=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
oc exec -n $NAMESPACE $POD1 -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep istio_requests_total | head -5
```

Query metrics from the second pod:

```
POD2=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[1].metadata.name}')
oc exec -n $NAMESPACE $POD2 -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep istio_requests_total | head -5
```

Compare the request counts between pods to see load distribution.

#### Step 3: Aggregate Metrics Across Pods

Get total request count across all pods:

```
for pod in $(oc get pods -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[*].metadata.name}'); do
  echo "Pod: $pod"
  oc exec -n $NAMESPACE $pod -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep
"istio_requests_total.*destination_service_name=\"metrics-test\"" | head -3
done
```

## Exercise 3: View Envoy Access Logs

In this exercise, you will view and analyze Envoy access logs directly from the sidecar containers.

#### Step 1: Generate Traffic with Different Patterns

Generate various types of traffic to create interesting logs:

```
# Successful requests
for i in {1..5}; do
  oc run curl-success-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://metrics-test > /dev/null
done

# Requests with different methods
oc run curl-post --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -s -X POST http://metrics-test > /dev/null
```

```
# Requests with headers
oc run curl-headers --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s -H "X-Test-Header: test-value" http://metrics-test > /dev/null
```

**Step 2: View Access Logs**

View the most recent access logs from a pod:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=20
```

The logs show detailed information about each request, including:
* Request method and path
* Response code
* Request and response sizes
* Duration
* Upstream service information

**Step 3: Filter Logs by Response Code**

View only successful requests (2xx status codes):

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=50 | grep "\" 200 "
```

View only error responses:

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=50 | grep -E "\" [45][0-9]{2} "
```

**Step 4: View Logs with Timestamps**

View logs with timestamps to see request timing:

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=20 --timestamps
```

**Step 5: Follow Logs in Real-Time**

Follow logs in real-time while generating traffic:

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy -f &
LOG_PID=$!

# Generate some traffic
for i in {1..3}; do
```

```
  oc run curl-realtime-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://metrics-test > /dev/null
  sleep 1
done

# Stop following logs
kill $LOG_PID
```

**Step 6: View Logs from Multiple Pods**

Compare logs from different pods:

```
for pod in $(oc get pods -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[*].metadata.name}'); do
  echo "=== Logs from $pod ==="
  oc logs -n $NAMESPACE $pod -c istio-proxy --tail=5
  echo ""
done
```

## Exercise 4: Analyze Request Patterns from Logs

In this exercise, you will analyze access logs to understand request patterns and service behavior.

**Step 1: Generate Diverse Traffic**

Generate traffic with different characteristics:

```
# Fast requests
for i in {1..10}; do
  oc run curl-fast-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://metrics-test > /dev/null
done

# Slow request (simulate with timeout)
oc run curl-slow --image=curlimages/curl:latest --rm -i --restart=Never -n $NAMESPACE
-- curl -s --max-time 5 http://metrics-test > /dev/null
```

**Step 2: Extract Request Durations**

Extract and analyze request durations from logs:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=50 | grep -oP 'duration="\K[0-
9]+' | head -20
```

#### Step 3: Count Requests by Response Code

Count requests by HTTP response code:

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=100 | grep -oP '"\s\K[0-9]{3}' |
sort | uniq -c
```

#### Step 4: Find Longest Requests

Find the requests with the longest duration:

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=100 | grep duration | sort -t'"'
-k4 -rn | head -5
```

#### Step 5: Analyze Request Sizes

Extract request and response sizes:

```
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=50 | grep -oP
'request_size="\K[0-9]+' | head -20
oc logs -n $NAMESPACE $POD_NAME -c istio-proxy --tail=50 | grep -oP
'response_size="\K[0-9]+' | head -20
```

### Exercise 5: Use istioctl for Observability

In this exercise, you will use `istioctl` commands to view proxy configuration and statistics.

#### Step 1: View Proxy Configuration

View the Envoy proxy configuration for a pod:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
istioctl proxy-config dump $POD_NAME -n $NAMESPACE | head -50
```

#### Step 2: View Proxy Stats

View Envoy proxy statistics using istioctl:

```
istioctl proxy-stats $POD_NAME -n $NAMESPACE
```

This shows a summary of key statistics.

**Step 3: View Specific Proxy Stats**

View specific statistics, such as request counts:

```
istioctl proxy-stats $POD_NAME -n $NAMESPACE | grep -i request
```

View connection statistics:

```
istioctl proxy-stats $POD_NAME -n $NAMESPACE | grep -i connection
```

**Step 4: View Cluster Information**

View cluster (upstream service) information:

```
istioctl proxy-config cluster $POD_NAME -n $NAMESPACE
```

This shows all the upstream services this pod can connect to.

**Step 5: View Listeners**

View listener configuration (what ports Envoy is listening on):

```
istioctl proxy-config listeners $POD_NAME -n $NAMESPACE
```

**Step 6: View Routes**

View route configuration:

```
istioctl proxy-config routes $POD_NAME -n $NAMESPACE
```

This shows how requests are routed to upstream services.

## Exercise 6: Monitor Metrics Over Time

In this exercise, you will monitor how metrics change over time by querying them periodically.

**Step 1: Create a Monitoring Script**

Create a simple script to monitor request counts:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')

for i in {1..5}; do
  echo "=== Sample $i at $(date) ==="
```

```
  oc exec -n $NAMESPACE $POD_NAME -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep
"istio_requests_total.*destination_service_name=\"metrics-test\"" | head -3
  echo ""
  sleep 5

  # Generate some traffic
  oc run curl-monitor-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://metrics-test > /dev/null
done
```

**Step 2: Monitor Error Rates**

Monitor error rates over time:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')

for i in {1..5}; do
  echo "=== Error Check $i at $(date) ==="
  oc exec -n $NAMESPACE $POD_NAME -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep
"istio_requests_total.*response_code=\"5" | head -3
  echo ""
  sleep 5
done
```

**Step 3: Compare Metrics Before and After Traffic**

Get baseline metrics:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
BASELINE=$(oc exec -n $NAMESPACE $POD_NAME -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep
"istio_requests_total.*destination_service_name=\"metrics-test\"" | head -1)
echo "Baseline: $BASELINE"
```

Generate traffic:

```
for i in {1..10}; do
  oc run curl-compare-$i --image=curlimages/curl:latest --rm -i --restart=Never -n
$NAMESPACE -- curl -s http://metrics-test > /dev/null
done
```

Check metrics after traffic:

```
AFTER=$(oc exec -n $NAMESPACE $POD_NAME -c istio-proxy -- curl -s
http://localhost:15020/stats/prometheus | grep
"istio_requests_total.*destination_service_name=\"metrics-test\"" | head -1)
echo "After traffic: $AFTER"
```

## Exercise 7: Explore Envoy Admin Interface

In this exercise, you will explore the Envoy admin interface to access additional observability information.

#### Step 1: Access Admin Interface

The Envoy admin interface is available on port 15000. Port-forward to access it:

```
POD_NAME=$(oc get pod -n $NAMESPACE -l app=metrics-test -o
jsonpath='{.items[0].metadata.name}')
oc port-forward -n $NAMESPACE $POD_NAME 15000:15000 &
```

#### Step 2: View Server Info

View Envoy server information:

```
curl http://localhost:15000/server_info
```

This shows Envoy version and build information.

#### Step 3: View Clusters

View cluster information via admin interface:

```
curl http://localhost:15000/clusters | head -50
```

This shows all upstream clusters with detailed statistics.

#### Step 4: View Listeners

View listener information:

```
curl http://localhost:15000/listeners
```

#### Step 5: View Config Dump

View the full configuration dump:

```
curl http://localhost:15000/config_dump | head -100
```

This provides a JSON representation of the entire Envoy configuration.

**Step 6: View Stats**

View statistics in JSON format:

```
curl http://localhost:15000/stats?format=json | head -100
```

**Step 7: Stop Port-Forwarding**

Stop the port-forward:

```
pkill -f "port-forward.*15000"
```

## Summary

In these exercises, you have:

- Accessed Prometheus-formatted metrics directly from Envoy sidecar proxies
- Queried metrics from multiple pods and compared load distribution
- Viewed and analyzed Envoy access logs to understand request patterns
- Used `istioctl` commands to view proxy configuration and statistics
- Monitored metrics over time to observe changes
- Explored the Envoy admin interface for additional observability data

These skills enable you to gain deep insights into your service mesh behavior using only the tools built into Istio and Envoy, without requiring external observability tools.

## Troubleshooting

If you encounter issues:

- **Cannot access metrics endpoint**: Verify the pod has a sidecar injected and check that port 15020 is accessible
- **No metrics appearing**: Ensure traffic has been generated to create metrics
- **Logs not showing**: Verify access logging is enabled in your Istio configuration
- **istioctl not working**: Ensure istioctl is installed and can connect to your cluster
- **Port-forwarding fails**: Check that the pod is running and the ports are not already in use
- **Admin interface not accessible**: Verify port 15000 is available and not blocked by network policies